

Simulation Tutorials

Lecture 5 Lecture Materials

Instructor: Takeshi Kawasaki

Osaka University, D3 center

Last update: May 23, 2026

Contents

- 1 Syllabus
- 2 Assignment 4, solution
- 3 Fifth Assignment
- 4 Appendix 1: Comparison of Stack Memory and Dynamic (Heap) Memory Allocation (C Language)
- 5 References

Contents

- 1 Syllabus
- 2 Assignment 4, solution
- 3 Fifth Assignment
- 4 Appendix 1: Comparison of Stack Memory and Dynamic (Heap) Memory Allocation (C Language)
- 5 References

1. Syllabus

The following topics are planned to be covered in this practical lecture (subject to change based on progress).

1 Introduction

- Using C(C++) (primarily for numerical calculations)
- Using Python (for data analysis and plotting)
- Principles of numerical computation
- Round-off errors
- Non-dimensionalization in scientific computing

2 Numerical Solutions of Ordinary Differential Equations: Examples of Damped Oscillations and Harmonic Oscillators

- Numerical integration of differential equations
- Stability of orbits and conservation laws

3 Brownian Motion of Single Particles

- Langevin Equation (Stochastic Differential Equation)
- Generating normal random numbers
- Euler-Maruyama method
- Time average and ensemble average

4 Brownian Motion of Multi-Particle Systems

- Calculation of interaction forces
- Simulation of non-equilibrium systems: Example of phase separation phenomena

5 Molecular Dynamics Simulation of Multi-Particle Systems

- Position Verlet method and velocity Verlet method
- Conservation laws in multi-particle systems

6 Monte Carlo Method

- Review of statistical mechanics
- Markov Chain Monte Carlo method
- Metropolis criterion

Contents

- 1 Syllabus
- 2 Assignment 4, solution**
- 3 Fifth Assignment
- 4 Appendix 1: Comparison of Stack Memory and Dynamic (Heap) Memory Allocation (C Language)
- 5 References

2. Assignment 4, solution

Fourth Assignment Implementation of Single-Particle Brownian Motion

Consider the motion of a single particle driven by thermal fluctuations in a three-dimensional solvent with temperature T and friction coefficient ζ . This particle's motion is widely known to be modeled by the Langevin equation $m\dot{\mathbf{v}}(t) = -\zeta\mathbf{v}(t) + \mathbf{F}_B(t)$. When this Langevin equation is non-dimensionalized using length a and time unit $\frac{m}{\zeta}$, the parameter $T^* = \frac{mk_BT}{a^2\zeta^2}$ becomes the main parameter. Answer the following questions about the motion of this particle. Note that **the notation $\langle \dots \rangle$ below refers to quantities averaged over time or ensemble.**

- (1) Non-dimensionalize the analytical solution for the mean square displacement $\langle \Delta \mathbf{r}(t)^2 \rangle = \frac{2dk_BT}{\zeta} \left\{ t + \frac{m}{\zeta} e^{-\zeta t/m} - \frac{m}{\zeta} \right\}$ (see the fourth lecture notes) and express it using the parameter T^* .
- (2) Confirm that the theoretical solution and numerical solution match for any T^* . The numerical solution should be obtained using the semi-implicit Euler-Maruyama method.
- (3) Compute the velocity autocorrelation function $C(t) = \langle \mathbf{v}(t) \cdot \mathbf{v}(0) \rangle$. Also, compare the numerical result with the theoretical solution $C(t) = \langle \mathbf{v}(t) \cdot \mathbf{v}(0) \rangle = \frac{dk_BT}{m} e^{-\zeta t/m}$.

2. Assignment 4, solution (2)

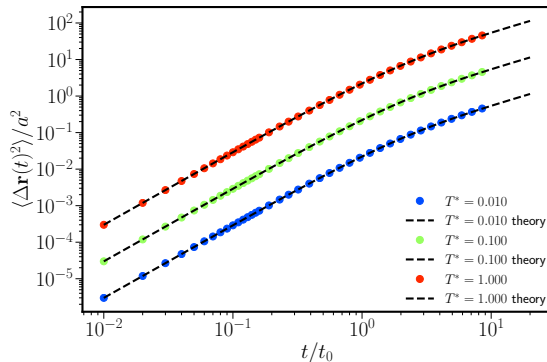


FIG 1: Numerical results of the mean square displacement. Here, the parameter T^* is varied as 0.01, 0.1, 1.0. The dashed lines represent the theoretical solution [Equation (3)]. The numerical solution agrees well with the theory.

2. Assignment 4, solution (3)

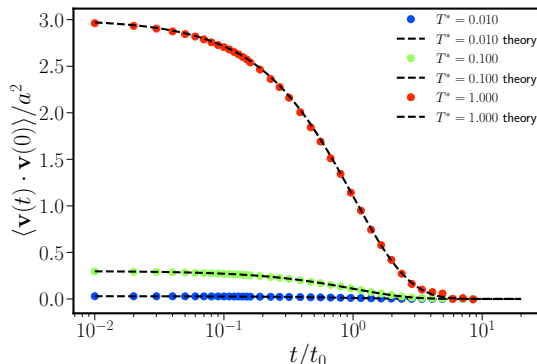


FIG 2: Numerical results of the velocity autocorrelation function. The parameter T^* is varied as 0.01, 0.1, 1.0. The dashed lines represent the theoretical solution [Equation (9)]. The numerical solution shows fairly good agreement.

2. Assignment 4, solution (4)

Solution

- (1) The analytical solution for the mean square displacement $\langle \Delta \mathbf{r}(t)^2 \rangle = \frac{2dk_{\text{B}}T}{\zeta} \left\{ t + \frac{m}{\zeta} e^{-\zeta t/m} - \frac{m}{\zeta} \right\}$ is non-dimensionalized. Considering the units of length a and time $t_0 = m/\zeta$, we obtain:

$$\begin{aligned} a^2 \langle \Delta \tilde{\mathbf{r}}(\tilde{t})^2 \rangle &= \frac{2dk_{\text{B}}T}{\zeta} [t_0 \tilde{t} + t_0 e^{-\tilde{t}} - t_0] \\ &= \frac{2dmk_{\text{B}}T}{\zeta^2} [\tilde{t} + e^{-\tilde{t}} - 1] \end{aligned} \quad (1)$$

Thus,

$$\begin{aligned} \langle \Delta \tilde{\mathbf{r}}(\tilde{t})^2 \rangle &= \frac{2dmk_{\text{B}}T}{\zeta^2 a^2} [\tilde{t} + e^{-\tilde{t}} - 1] \\ &= 2dT^* [\tilde{t} + e^{-\tilde{t}} - 1] \end{aligned} \quad (2)$$

Therefore, in three dimensions ($d = 3$):

$$\boxed{\langle \Delta \tilde{\mathbf{r}}(\tilde{t})^2 \rangle = 6T^* [\tilde{t} + e^{-\tilde{t}} - 1]} \quad (3)$$

2. Assignment 4, solution (5)

is obtained.

Supplement

In the short-time limit, by expanding $e^{-\tilde{t}} \approx 1 - \tilde{t} + \frac{1}{2}\tilde{t}^2$:

$$\langle \Delta \tilde{\mathbf{r}}(\tilde{t})^2 \rangle \approx 3T^* \tilde{t}^2 \quad (4)$$

a ballistic trajectory is obtained. In the long-time limit, the term $e^{-\tilde{t}} - 1$ drops out, so:

$$\langle \Delta \tilde{\mathbf{r}}(\tilde{t})^2 \rangle \approx 6T^* \tilde{t} \quad (5)$$

a diffusive trajectory is obtained. Thus, the non-dimensionalized diffusion coefficient is equal to the value of the parameter T^* itself.

- (2) Examples of numerical computation in C language are provided in Listings 2 and 3. The main computation program that solves the equation of motion is “langevin.cpp,” and the analysis program for the mean square displacement and velocity autocorrelation function, using the data output from “langevin.cpp,” is “analyze.cpp.” Numerical results for the mean square displacement are shown in Fig. 1. The parameter T^* is varied as 0.01, 0.1, 1.0. The dashed lines represent the theoretical solution [Equation (3)]. Good agreement with the theory is observed.

2. Assignment 4, solution (6)

- (3) As shown in the supplement of the fourth lecture, the velocity autocorrelation function of a Brownian particle for $t \geq 0$ is given by:

$$C(t) = \langle \mathbf{v}(t) \cdot \mathbf{v}(0) \rangle = \frac{dk_B T}{m} e^{-\zeta t/m} \quad (6)$$

When non-dimensionalized, this becomes:

$$\frac{a^2}{t_0^2} \langle \tilde{\mathbf{v}}(\tilde{t}) \cdot \tilde{\mathbf{v}}(0) \rangle = \frac{dk_B T}{m} e^{-\zeta t/m} \quad (7)$$

Thus,

$$\langle \tilde{\mathbf{v}}(\tilde{t}) \cdot \tilde{\mathbf{v}}(0) \rangle = \frac{dmk_B T}{\zeta^2 a^2} e^{-\zeta t/m} \quad (8)$$

$$= 3T^* e^{-\tilde{t}} \quad (9)$$

where $d = 3$ is assumed. A comparison between this theoretical solution and numerical solution is shown in Fig. 2. The numerical solution shows fairly good agreement with the theory, though not as precise as for the mean square displacement.

2. Assignment 4, solution (7)

LIST 1: "langevin.cpp" Available from the following GitHub repository [\[Link\]](#)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4  #include <iostream>
5  #include <fstream>
6  #include <cfloat>
7  #include "BM.h"
8
9  #define tmax 10
10 #define dt 0.01
11 #define temp 0.01 //parameter
12 #define ensemble 1000
13 #define dim 3
14 //using namespace std;
15
16 void ini_phase(double *x, double *v){
17     int i;
18     for(i=0; i<dim; i++){
19         x[i]=0.;
20         v[i]=0.;
21     }
22 }
```

2. Assignment 4, solution (8)

```

23
24 void ini_clock(int *j, double *tout){
25     *j=0;
26     *tout=1.e-2;
27 }
28
29 void eom(double *v, double *x){
30     int i;
31     for(i=0; i<dim; i++){
32         v[i]+=-v[i]*dt+sqrt(2.*temp*dt)*gaussian_rand();
33         x[i]+=v[i]*dt;
34     }
35 }
36
37 void output(double *x, double *v, int j){
38     char filename[128];
39     std::ofstream file;
40
41     sprintf(filename, "coord_dt%.3fT%.3f.dat", dt, temp);
42     file.open(filename, std::ios::app); //append
43     file <<j*dt<<"\t"<<x[0]<<"\t"<<x[1]<<"\t"<<x[2]<<std::endl;
44     // std::cout<<j*dt<<"\t"<<x[0]<<"\t"<<x[1]<<"\t"<<x[2]<<std::endl;
45     file.close();
46

```

2. Assignment 4, solution (9)

```

47  sprintf(filename, "vel_dt%.3fT%.3f.dat", dt, temp);
48  file.open(filename, std::ios::app); //append
49  file << j*dt << "\t" << v[0] << "\t" << v[1] << "\t" << v[2] << std::endl;
50  file.close();
51
52  }
53
54  int main(){
55      double x[dim], v[dim], t, tout;
56      int i, j;
57      ini_phase(x, v);
58      for(i=0; i<ensemble; i++){
59          ini_clock(&j, &tout);
60          output(x, v, j);
61          while(j*dt < tmax){
62              j++;
63              eom(v, x);
64              if(j*dt >= tout){
65                  output(x, v, j);
66                  tout*=1.2;
67              }
68          }
69      }
70      return 0;

```

2. Assignment 4, solution (10)

71 }

LIST 2: “analyze.cpp” Available from the following GitHub repository [\[Link\]](#)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4  #include <iostream>
5  #include <fstream>
6  #include <cfloat>
7
8  #define temp 0.01
9  #define dt 0.01
10 #define ensemble 1000
11 #define window 39
12 #define dim 3
13 //using namespace std;
14
15 void ini(double *dr2, double *corr){
16     for(int i=0; i<window; i++){
17         dr2[i]=0.0;
18         corr[i]=0.0;
19     }
```

2. Assignment 4, solution (11)

```

20 }
21
22 void input(double (*x)[dim],double (*v)[dim],double *t){
23     char filename[128];
24     std::ifstream file;
25     sprintf(filename,"coord_dt%.3fT%.3f.dat",dt,temp);
26     file.open(filename);
27     int asize=ensemble*window;
28     for(int i=0;i<asize;i++){
29         file >> t[i] >> x[i][0] >> x[i][1] >> x[i][2];
30     }
31     file.close();
32
33     sprintf(filename,"vel_dt%.3fT%.3f.dat",dt,temp);
34     file.open(filename);
35     for(int i=0;i<asize;i++){
36         file >> t[i] >> v[i][0] >> v[i][1] >> v[i][2];
37         // std::cout << t[i] <<"\t"<<v[0][i]<<"\t"<<v[1][i]<<"\t"<<v[2][i]<<std::endl;
38     }
39     file.close();
40 }
41
42 void output(double *t,double *dr2,double *corr){
43     char filename[128];

```

2. Assignment 4, solution (12)

```

44     std::ofstream file;
45     sprintf(filename, "msd_dt%.3fT%.3f.dat", dt, temp);
46     file.open(filename);
47     for(int i=1; i<window; i++)
48         file<<t[i]-t[0]<<"\t"<<dr2[i]<<std::endl;
49     file.close();
50
51     sprintf(filename, "corr_dt%.3fT%.3f.dat", dt, temp);
52     file.open(filename);
53     for(int i=1; i<window; i++)
54         file<<t[i]-t[0]<<"\t"<<corr[i]<<std::endl;
55     file.close();
56 }
57
58 void analyze(double (*x)[dim], double (*v)[dim], double *t, double *dr2, double *corr){
59     double dx[dim], corr_x[dim];
60     for(int i=0; i<ensemble; i++)
61         for(int j=0; j<window; j++){
62             for(int k=0; k<dim; k++){
63                 dx[k]=(x[j+window*i][k]-x[window*i][k]);
64                 corr_x[k]=v[j+window*i][k]*v[window*i][k];
65                 dr2[j]+=(dx[k]*dx[k])/ensemble;
66                 corr[j]+=(corr_x[k])/ensemble;
67             }

```

2. Assignment 4, solution (13)

```
68     }
69 }
70
71 int main(){
72     double t[ensemble*window], dr2[window], corr[window];
73     int i, j;
74     double (*x)[dim] = new double[ensemble*window][dim];
75     double (*v)[dim] = new double[ensemble*window][dim];
76
77     ini(dr2, corr);
78     input(x, v, t);
79     analyze(x, v, t, dr2, corr);
80     output(t, dr2, corr);
81     delete[] x;
82     delete[] v;
83     return 0;
84 }
```

Contents

- 1 Syllabus
- 2 Assignment 4, solution
- 3 Fifth Assignment**
- 4 Appendix 1: Comparison of Stack Memory and Dynamic (Heap) Memory Allocation (C Language)
- 5 References

3. Fifth Assignment

Fifth Assignment Preparation for Multi-Particle Calculations

In a two-dimensional plane, distribute 512 discs with a diameter of 1 in a square space with a side length of $L = 40$.

- (1) Arrange the particles in a square lattice and visualize the result.
- (2) Arrange the particles in a hexagonal lattice and visualize the result.
- (3) (Advanced) Consider an algorithm for measuring the distance between particle i and other particles j when the square boundary is periodic (necessary for force calculations, etc.).
- (4) (Advanced) Consider an algorithm for storing the "particle numbers" within a distance (e.g., 5) calculated in (3) into an array (Verlet list).

Contents

- 1 Syllabus
- 2 Assignment 4, solution
- 3 Fifth Assignment
- 4 Appendix 1: Comparison of Stack Memory and Dynamic (Heap) Memory Allocation (C Language)
- 5 References

4. Appendix 1: Comparison of Stack Memory and Dynamic (Heap) Memory Allocation (C Language)

LIST 3: Variable and Array Allocation Using Stack Memory (Example in C). Stack memory (in the stack region) is generally limited to around 10 MB on a laptop PC (quite small) [3].

```
1 double x, y[10000], z[10000][10];
```

LIST 4: Dynamic Memory Allocation (Example in C++). The method for releasing memory differs between variables and arrays. Memory (in the heap region) can generally be allocated up to the GB order on a laptop PC [3]. The same can be done in C language using the malloc function.

```
1 double *x = new double;  
2 double *y = new double[10000];  
3 double (*z)[10] = new double[10000][10];  
4 // Free memory when no longer needed  
5 delete x;  
6 delete [] y;  
7 delete [] z;
```

Contents

- 1 Syllabus
- 2 Assignment 4, solution
- 3 Fifth Assignment
- 4 Appendix 1: Comparison of Stack Memory and Dynamic (Heap) Memory Allocation (C Language)
- 5 References**

References and Websites

- [1] Box GEP, Muller ME (1958) A Note on the Generation of Random Normal Deviates.
The Annals of Mathematical Statistics 29(2):610–611.
- [2] Marsaglia G, Bray TA (1964) A Convenient Method for Generating Normal Variables.
SIAM Review 6(3):260–264.
- [3] Lemniscater N (year?) C++のスタックメモリと動的メモリの上限値調査
(<https://qiita.com/LemniscaterN/items/a3abfa143612cb928bde>).